

Arquitectura de Pagos — Vision Completa

TaxVision.PaymentApp • TaxVision.PaymentClient • Extensibilidad

Basado en analisis de CloudTax + diseno extensible para ~25 microservicios

Autor: Jorge Turbi | Fecha: 02-07-2026 | Version 2.0

1. Por Que Esta Version del Plan

La version anterior documentaba la separacion de los dos contextos de pago, pero no respondia dos preguntas criticas: **(a) como cualquier microservicio futuro puede solicitar un pago sin que PaymentApp tenga que ser modificado cada vez**, y **(b) como se agrega un nuevo metodo de pago** a nivel de plataforma o de tenant. Esta version responde ambas con diseno concreto.

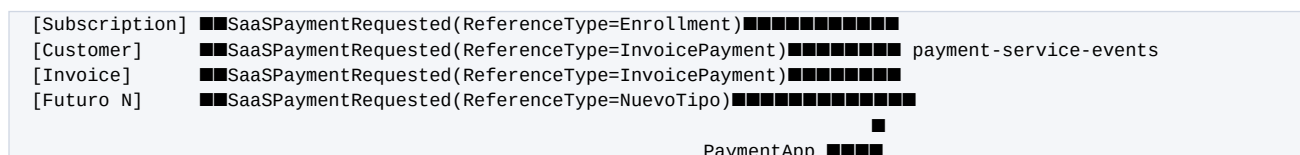
Bug critico encontrado en el diseno actual: el webhook handler llama MarkCompleted/MarkFailed en SaaSPayment pero NO re-publica los eventos de resultado a RabbitMQ. En produccion (donde Stripe confirma via webhook, no sincronamente) Subscription NUNCA recibe la notificacion y el flujo queda colgado. Esto se corrige en esta version.

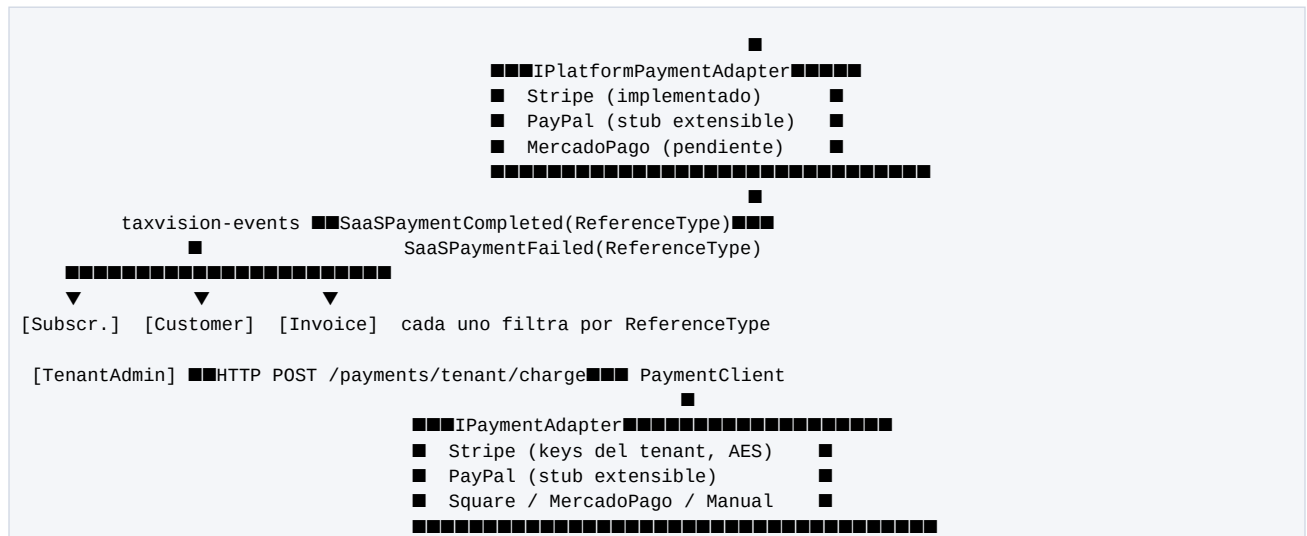
2. Mapa Completo del Sistema de Pagos

TaxVision tiene aproximadamente 25 microservicios planificados. Varios de ellos necesitaran pagos. El mapa muestra todos los flujos actuales y futuros.

Microservicio	Tipo de pago	Contexto	Como se activa	Cola
Subscription	Enrollment, compra/renovacion de asientos, renovacion de suscripcion	PaymentApp	Publica SaaSPaymentRequested IntegrationEvent	payment-servic e-events
Customer (futuro)	Cobro de factura al cliente del tenant	PaymentApp	Publica SaaSPaymentRequested IntegrationEvent	payment-servic e-events
Invoice (futuro)	Pago de factura emitida	PaymentApp	Publica SaaSPaymentRequested IntegrationEvent	payment-servic e-events
Cualquier servicio futuro	Cualquier cobro de la plataforma al tenant	PaymentApp	Publica el mismo evento con nuevo ReferenceType	payment-servic e-events
TenantAdmin (cualquier servicio)	El tenant cobra a sus propios clientes	PaymentClient	HTTP POST /payments/tenant/charge	ninguna — HTTP puro

Flujo de eventos — vision completa:





3. Patron Generico de Eventos — Diseno Escalable

El problema del diseno actual: cada nuevo tipo de pago requiere 3 clases de eventos nuevas + 1 handler nuevo en PaymentApp. Para 25 microservicios eso es potencialmente 75 clases de eventos. La solucion es un patron generico con un discriminador **ReferenceType**.

3.1 Eventos genericos en BuildingBlocks (reemplaza 8 eventos actuales)

```
// BuildingBlocks/Messaging/SaaSPaymentRequestedIntegrationEvent.cs
// UN SOLO evento para TODOS los servicios que necesiten pago de plataforma
public sealed record SaaSPaymentRequestedIntegrationEvent : IIntegrationEvent
{
    public Guid      EventId      { get; init; } = Guid.NewGuid();
    public Guid      TenantId      { get; init; }
    public DateTime   OccurredOn    { get; init; } = DateTime.UtcNow;
    public string     CorrelationId { get; init; } = string.Empty;
    public Guid      ReferenceId    { get; init; } // EnrollmentId, SeatId, InvoiceId...
    public PaymentReferenceType ReferenceType { get; init; } // discriminador
    public long       AmountCents   { get; init; }
    public string     Currency      { get; init; } = "USD";
    public string?    Description   { get; init; }
}

// BuildingBlocks/Messaging/SaaSPaymentCompletedIntegrationEvent.cs
public sealed record SaaSPaymentCompletedIntegrationEvent : IIntegrationEvent
{
    public Guid      EventId      { get; init; } = Guid.NewGuid();
    public Guid      TenantId      { get; init; }
    public DateTime   OccurredOn    { get; init; } = DateTime.UtcNow;
    public string     CorrelationId { get; init; } = string.Empty;
    public Guid      ReferenceId    { get; init; }
    public PaymentReferenceType ReferenceType { get; init; }
    public string     StripePaymentIntentId { get; init; } = string.Empty;
}

// BuildingBlocks/Messaging/SaaSPaymentFailedIntegrationEvent.cs
public sealed record SaaSPaymentFailedIntegrationEvent : IIntegrationEvent
{
    public Guid      EventId      { get; init; } = Guid.NewGuid();
    public Guid      TenantId      { get; init; }
    public DateTime   OccurredOn    { get; init; } = DateTime.UtcNow;
    public string     CorrelationId { get; init; } = string.Empty;
    public Guid      ReferenceId    { get; init; }
    public PaymentReferenceType ReferenceType { get; init; }
    public string     Reason        { get; init; } = string.Empty;
}

// BuildingBlocks/Enums/PaymentReferenceType.cs
public enum PaymentReferenceType
{
    Enrollment,          // ya existe
    SeatPurchase,        // ya existe
    SeatRenewal,         // ya existe
    SubscriptionRenewal, // ya existe
    InvoicePayment,       // futuro - CustomerService / Invoice
    CustomerDeposit,     // futuro
    // agregar aqui sin tocar PaymentApp
}
```

3.2 PaymentApp — UN handler generico para todos los servicios

```
// PaymentApp.Application/SaaSPayments/Handlers/SaaSPaymentRequestedHandler.cs
public static class SaaSPaymentRequestedHandler
{
    public static async Task Handle(
```

```

        SaaSPaymentRequestedIntegrationEvent evt,
        ISaaSPaymentRepository payments,
        IStripeCustomerRepository customers,
        IPlatformPaymentAdapterFactory adapterFactory,    // resuelve proveedor configurado
        IUnitOfWork uow,
        IMessageBus bus,
        ICorrelationContext correlation,
        CancellationToken ct)
    {
        using var _ = correlation.Push(evt.CorrelationId);

        var adapter = adapterFactory.GetAdapter(); // Stripe, PayPal, etc. por config
        var customer = await adapter.GetOrCreateCustomerAsync(evt.TenantId, ct);

        var payment = SaaSPayment.Create(
            evt.TenantId, evt.ReferenceType, evt.ReferenceId,
            evt.AmountCents, evt.Currency);

        var intentId = await adapter.CreatePaymentIntentAsync(
            customer.ExternalId, evt.AmountCents, evt.Currency, ct);
        payment.MarkProcessing(intentId);

        await uow.SaveChangesAsync(ct);
        // La confirmacion ocurre via webhook – no sincrona en produccion
    }
}

```

3.3 Webhook — re-publica el evento de resultado (bug fix)

```

// PaymentApp.Application/SaaSPayments/Handlers/ProcessSaaSPaymentHandler.cs
public static class ProcessSaaSPaymentHandler
{
    public static async Task<Result> Handle(
        ProcessSaaSPaymentCommand command,
        ISaaSPaymentRepository payments,
        IUnitOfWork uow,
        IMessageBus bus,
        ICorrelationContext correlation,
        CancellationToken ct)
    {
        var payment = await payments.GetByStripePaymentIntentIdAsync(
            command.StripePaymentIntentId, ct);
        if (payment is null)
            return Result.Failure(new Error("Payment.NotFound", "..."));

        using var _ = correlation.Push(payment.CorrelationId);

        if (command.IsSuccess)
        {
            payment.MarkCompleted();
            await uow.SaveChangesAsync(ct);
            // RE-PUBLICA para que el servicio origen lo reciba
            await bus.PublishAsync(new SaaSPaymentCompletedIntegrationEvent
            {
                TenantId = payment.TenantId,
                ReferenceId = payment.ReferenceId,
                ReferenceType = payment.ReferenceType,
                StripePaymentIntentId = command.StripePaymentIntentId,
                CorrelationId = payment.CorrelationId,
            });
        }
        else
        {
            payment.MarkFailed(command.FailureReason ?? "Webhook failure.");
            await uow.SaveChangesAsync(ct);
            await bus.PublishAsync(new SaaSPaymentFailedIntegrationEvent

```

```

        {
            TenantId = payment.TenantId,
            ReferenceId = payment.ReferenceId,
            ReferenceType = payment.ReferenceType,
            Reason = command.FailureReason ?? "Webhook failure.",
            CorrelationId = payment.CorrelationId,
        });
    }
    return Result.Success();
}
}

```

3.4 Como consume un microservicio el resultado

```

// Subscription.Application/Handlers/SaaSPaymentCompletedHandler.cs
public static class SaaSPaymentCompletedHandler
{
    public static async Task Handle(
        SaaSPaymentCompletedIntegrationEvent evt,
        ISubscriptionRepository subs,
        IEnrollmentRepository enrollments,
        ISeatRepository seats,
        IUnitOfWork uow,
        IMessageBus bus,
        Cancellation token ct)
    {
        switch (evt.ReferenceType) // solo filtra los tipos que le pertenecen
        {
            case PaymentReferenceType.Enrollment:
                // completar enrollment, provisionar tenant
                break;
            case PaymentReferenceType.SeatPurchase:
                // activar asientos
                break;
            case PaymentReferenceType.SeatRenewal:
            case PaymentReferenceType.SubscriptionRenewal:
                // renovar
                break;
            // InvoicePayment, CustomerDeposit -> Subscription los ignora
        }
    }
}

```

Customer e Invoice consumen el mismo SaaSPaymentCompletedIntegrationEvent y filtran ReferenceType.InvoicePayment. PaymentApp no sabe ni le importa quien consume.

4. Como Agregar Nuevos Metodos de Pago

Hay dos niveles completamente independientes de extensibilidad de metodos de pago.

4.1 Nivel Plataforma — PaymentApp (TaxVision cobra a tenants)

PaymentApp usa **IPlatformPaymentAdapter** para procesar cobros de la plataforma. La implementacion actual (StripeGateway) se convierte en el primer adaptador. Agregar PayPal o MercadoPago como alternativa de plataforma solo requiere implementar la interfaz y registrarla — sin cambiar ningun handler ni evento.

```
// PaymentApp.Domain/Abstractions/IPlatformPaymentAdapter.cs
public interface IPlatformPaymentAdapter
{
    PlatformPaymentProvider Provider { get; }

    Task<PlatformCustomerResult> GetOrCreateCustomerAsync(
        Guid tenantId, CancellationToken ct);

    Task<string> CreatePaymentIntentAsync(
        string externalCustomerId, long amountCents,
        string currency, CancellationToken ct);

    Task<bool> VerifyWebhookSignatureAsync(
        string payload, string signature, CancellationToken ct);
}

public enum PlatformPaymentProvider { Stripe, PayPal, MercadoPago }

// IPlatformPaymentAdapterFactory – resuelve por env var PLATFORM_PAYMENT_PROVIDER
public interface IPlatformPaymentAdapterFactory
{
    IPlatformPaymentAdapter GetAdapter();
}

// Para agregar PayPal como proveedor de plataforma:
// 1. Implementar PayPalPlatformAdapter : IPlatformPaymentAdapter
// 2. Registrar en AddPaymentAppInfrastructure
// 3. Cambiar PLATFORM_PAYMENT_PROVIDER=PayPal en .env
// Sin tocar handlers, sin tocar eventos.
```

4.2 Nivel Tenant — PaymentClient (el tenant cobra a sus clientes)

PaymentClient ya tiene **IPaymentAdapter** e **IPaymentAdapterFactory**. Cada tenant configura su propio proveedor via POST /payments/tenant/config. Agregar un nuevo proveedor solo requiere implementar IPaymentAdapter.

Proveedor	Estado	Como activar
Stripe	Implementado	SecretKey del tenant (AES-cifrado). Funciona ahora.
PayPal	Stub — pendiente	Implementar PayPal REST Orders API en PayPalPaymentAdapter
Square	Enum declarado	Crear SquarePaymentAdapter : IPaymentAdapter + registrar
MercadoPago	Enum declarado	Crear MercadoPagoPaymentAdapter : IPaymentAdapter + registrar
Manual	Pendiente	ManualPaymentAdapter — registra pago sin llamada externa

```
// Para agregar Square como metodo de pago del tenant:
// 1. Agregar Square al enum TenantPaymentProvider
// 2. Crear SquarePaymentAdapter : IPaymentAdapter en PaymentClient.Infrastructure
// 3. Registrar en DependencyInjection.AddPaymentClientInfrastructure
// El TenantAdmin ya puede seleccionar Square en POST /payments/tenant/config
```

4.3 Tabla comparativa de extensibilidad

Escenario	Que se necesita agregar	Que NO cambia
Nuevo servicio usa PaymentApp	Valor en PaymentReferenceType enum Consumidor del evento en su propio servicio	PaymentApp (ningun handler, ningun evento nuevo) BuildingBlocks (solo el enum)
Nuevo proveedor plataforma (PayPal SaaS)	PayPalPlatformAdapter : IPlatformPaymentAdapter Registro en DI Cambiar env var PLATFORM_PAYMENT_PROVIDER	Todos los handlers Todos los eventos Gateway Docker-compose
Nuevo proveedor tenant (Square)	SquarePaymentAdapter : IPaymentAdapter Registro en DI	Controladores Handlers Eventos Configuración

5. TaxVision.PaymentApp — Estructura Completa

Cambios respecto al plan anterior: un solo handler generico reemplaza los 4 handlers especificos; StripeGateway se convierte en StripePaymentAdapter implementando IPlatformPaymentAdapter; SaaSPayment agrega CorrelationId; se corrige el webhook.

5.1 Capas

Capa	Contenido
Domain	SaaSPayment (sealed, private ctor, factory Create, MarkProcessing/Completed/Failed) + campo CorrelationId para propagacion en webhook StripeCustomer (sealed) IPlatformPaymentAdapter, IPlatformPaymentAdapterFactory Enums: PaymentReferenceType (antes SaaSPaymentType), PaymentStatus, PlatformPaymentProvider
Application — Abstractions	ISaaSPaymentRepository (incl. GetByStripePaymentIntentIdAsync) IStripeCustomerRepository (o IExternalCustomerRepository — generico)
Application — Handlers	SaaSPaymentRequestedHandler (static) — UN handler para todos los ReferenceType ProcessSaaSPaymentHandler (static) — webhook, ahora RE-PUBLICA el evento de resultado
Application — Queries	GetSaaSPaymentHandler, GetSaaSPaymentsByTenantHandler
Infrastructure	StripePlatformAdapter : IPlatformPaymentAdapter (renombrado de StripeGateway) PlatformPaymentAdapterFactory : IPlatformPaymentAdapterFactory SaaSPaymentRepository, ExternalCustomerRepository PaymentAppDbContext (sealed, ConflictException) DI.AddPaymentAppInfrastructure
Api	SaaSPaymentsController (GET /payments/saas/{id}, GET /payments/saas) WebhooksController (POST /webhooks/stripe — se generaliza a /webhooks/{provider}) Program.cs con Wolverine outbox+inbox, retry 1s/5s/15s

5.2 Base de datos — TaxVision_PaymentApp

Tabla	Campos adicionales vs disenio anterior	Indices
SaaSPayments	ReferenceType (antes SaaSPaymentType — mismo enum, renombrado) CorrelationId — para propagacion en webhook	IX_SaaSPayments_StripePaymentIntentId (unico) IX_SaaSPayments_TenantId
ExternalCustomers	Provider — para soportar multiples proveedores ExternalId (antes StripeCustomerId)	IX_ExternalCustomers_TenantId_Provider (unico)
wolverine_*	Sin cambio	—

6. TaxVision.PaymentClient — Estructura Completa

Sin cambios estructurales respecto al plan anterior. Se agrega AesEncryptionService implementado y se detalla como se extiende para nuevos proveedores.

6.1 Capas

Capa	Contenido
Domain	TenantPaymentConfig (sealed, private ctor, factory Create, Configure, Deactivate) TenantTransaction (sealed, private ctor, factory Create, MarkCompleted, MarkFailed) IPaymentAdapter, IPaymentAdapterFactory Enum: TenantPaymentProvider (Stripe, PayPal, Square, MercadoPago, Manual)
Application	ITenantPaymentConfigRepository, ITenantTransactionRepository IAesEncryptionService — interfaz declarada en Application (no en Domain) ConfigureTenantProviderHandler (static) ProcessTenantPaymentHandler (static)
Infrastructure	StripePaymentAdapter, PayPalPaymentAdapter (stub), PaymentAdapterFactory AesEncryptionService : IAesEncryptionService (AES-256-CBC, key desde env var) TenantPaymentConfigRepository, TenantTransactionRepository PaymentClientDbContext (sealed, ConflictException)

Capa	Contenido
Api	TenantPaymentsController: GET/POST /payments/tenant/config, POST /payments/tenant/charge GET /payments/tenant/transactions Program.cs (sin RabbitMQ, sin outbox/inbox)

7. Receta: Nuevo Microservicio que Usa Pagos

Con el nuevo diseño, agregar pagos a cualquier microservicio futuro (Customer, Invoice, TaxReport, etc.) requiere exactamente **4 pasos**, sin tocar PaymentApp ni BuildingBlocks (salvo el enum).

Pa so	Que hacer	Donde
1	Agregar el valor al enum PaymentReferenceType Ejemplo: InvoicePayment	BuildingBlocks/Enums/PaymentReferenceType.cs
2	Publicar SaaSPaymentRequestedIntegrationEvent con el nuevo ReferenceType y el ReferenceId del objeto de negocio (InvoiceId, etc.)	El nuevo microservicio — Application layer, en el handler que inicia el cobro
3	Configurar Wolverine en el nuevo servicio para escuchar SaaSPaymentCompletedIntegrationEvent y SaaSPaymentFailedIntegrationEvent en su propia cola (invoice-service-events)	Program.cs del nuevo microservicio
4	Crear el handler del resultado que filtre por ReferenceType y ejecute la logica de negocio correspondiente	El nuevo microservicio — Application/Handlers/SaaSPaymentCompletedHandler.cs

PaymentApp NO necesita saber que existe el nuevo servicio. El enum PaymentReferenceType en BuildingBlocks es el unico punto de extension compartido.

8. Infraestructura — docker-compose, YARP, .env

8.1 Variables de entorno (.env.example)

```
# PaymentApp
PAYMENT_APP_DB_CONNECTION=Server=host.docker.internal,1433;Database=TaxVision_PaymentApp;...
STRIPE_SECRET_KEY=sk_live_... # keys de la PLATAFORMA – nunca en appsettings.json
STRIPE_WEBHOOK_SECRET=whsec_...
PLATFORM_PAYMENT_PROVIDER=Stripe # cambiar a PayPal/MercadoPago cuando se implementen
RABBITMQ_CONNECTION=amqp://...

# PaymentClient
PAYMENT_CLIENT_DB_CONNECTION=Server=host.docker.internal,1433;Database=TaxVision_PaymentClient;...
AES_ENCRYPTION_KEY=replace-with-32-byte-base64-key # cifrado de keys del tenant

# Compartidas
JWT_SECRET=...

# ELIMINADAS
# PAYMENT_DB_CONNECTION – ya no existe como BD monolitica
```

8.2 YARP Gateway

Ruta	Path Match	Cluster	Auth
payment-saas	/payments/saas/{**catch-all}	payment-app	PlatformAdmin
webhooks	/webhooks/{**catch-all}	payment-app	Anonimo (firma verificada)
payment-tenant	/payments/tenant/{**catch-all}	payment-client	TenantAdmin

9. Migracion de Base de Datos

Tabla actual (TaxVision_Payment)	Destino	Accion
SaaSPayments	TaxVision_PaymentApp	Migrar esquema + renombrar SaaSPaymentType -> ReferenceType, agregar CorrelationId
StripeCustomers	TaxVision_PaymentApp	Migrar a ExternalCustomers + agregar columna Provider='Stripe'
TenantPaymentConfigs	TaxVision_PaymentClient	INSERT SELECT desde TaxVision_Payment
TenantTransactions	TaxVision_PaymentClient	INSERT SELECT desde TaxVision_Payment
wolverine_*	TaxVision_PaymentApp	Solo PaymentApp usa outbox/inbox

10. Orden de Implementacion (22 pasos)

#	Tarea	Prereq.
1	Crear carpetas src/Services/PaymentApp/ y src/Services/PaymentClient/	—
2	Crear los 8 proyectos .csproj (4 capas x 2 servicios)	1
3	Actualizar BuildingBlocks: reemplazar 8 eventos especificos por 3 genericos + PaymentReferenceType enum	2
4	Crear PaymentApp.Domain: SaaSPayment (con ReferenceType y CorrelationId), ExternalCustomer, IPlatformPaymentAdapter	2
5	Crear PaymentClient.Domain: TenantPaymentConfig, TenantTransaction, IPaymentAdapter, IPaymentAdapterFactory	2
6	Crear Application layer PaymentApp: ISaaSPaymentRepository, handler generico, webhook handler (con re-publicacion)	4
7	Crear Application layer PaymentClient: IAesEncryptionService, repositorios, handlers	5
8	Crear PaymentAppDbContext (sealed, ConflictException, ExternalCustomers con Provider)	4,6
9	Crear PaymentClientDbContext (sealed, ConflictException)	5,7
10	Implementar StripePlatformAdapter : IPlatformPaymentAdapter y PlatformPaymentAdapterFactory	8
11	Implementar AesEncryptionService (AES-256-CBC, key desde env var)	9
12	Implementar StripePaymentAdapter (tenant), PayPalPaymentAdapter (stub), PaymentAdapterFactory	9,11
13	Implementar SaaSPaymentRepository, ExternalCustomerRepository	8
14	Implementar TenantPaymentConfigRepository, TenantTransactionRepository	9
15	Crear migracion inicial PaymentApp (dotnet ef migrations add Initial)	8,13
16	Crear migracion inicial PaymentClient (dotnet ef migrations add Initial)	9,14
17	Crear Controllers y Program.cs en PaymentApp.Api (Wolverine outbox+inbox)	6,8
18	Crear Controllers y Program.cs en PaymentClient.Api (sin RabbitMQ)	7,9
19	Crear Dockerfiles para ambos servicios	17,18
20	Actualizar docker-compose, Gateway YARP, .env.example	19
21	Actualizar Subscription para consumir eventos genericos (filtrar por ReferenceType)	3
22	Eliminar TaxVision.Payment.*, ejecutar migracion SQL, actualizar CLAUDE.md/README.md	15..21

11. Reglas Criticas

- **STRIPE_SECRET_KEY** y **STRIPE_WEBHOOK_SECRET** son de la PLATAFORMA. Solo PaymentApp. NUNCA en appsettings.json.
- **Keys del tenant** en TenantPaymentConfig se almacenan AES-256 cifradas. NUNCA expuestas en HTTP.
- **PaymentClient NO usa RabbitMQ** — sin outbox, sin inbox. HTTP-driven puro.
- **PaymentApp NO conoce** TenantPaymentConfig ni IPaymentAdapter de tenant.
- **PaymentClient NO conoce** SaaSPayment ni IPlatformPaymentAdapter.

- **StripePlatformAdapter en PaymentApp** usa keys de la PLATAFORMA. StripePaymentAdapter en PaymentClient usa keys del TENANT.
- **El webhook handler DEBE re-publicar** SaaSPaymentCompleted/Failed despues de MarkCompleted/MarkFailed.
- **SaaSPayment.CorrelationId** se guarda en Create y se usa al re-publicar en el webhook.
- **Todos los handlers son static class con static Handle** (patron Wolverine de TaxVision).
- **No hay handlers en Infrastructure** — solo repositorios e implementaciones tecnicas.
- **Nuevo servicio que necesite pagos:** agrega ReferenceType al enum + publica el evento generico + consume el resultado. Sin tocar PaymentApp.
- **Nuevo proveedor de plataforma:** implementa IPlatformPaymentAdapter + registra + cambia env var. Sin tocar handlers.
- **Nuevo proveedor de tenant:** implementa IPaymentAdapter + registra. Sin tocar controladores.
- **Rama git:** feature/payment-split desde feature/payment-service.

Este documento requiere aprobacion antes de iniciar cualquier implementacion.